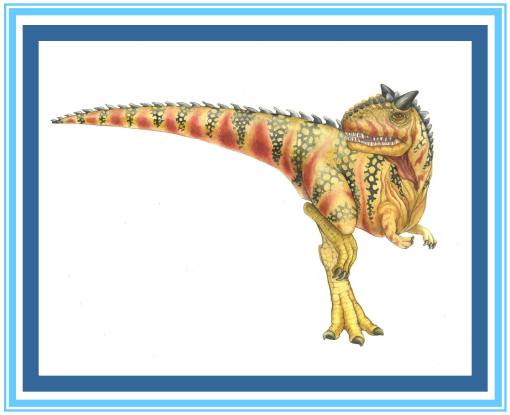
Chapter 11: File-System Implementation



Silberschatz, Galvin and Gagne ©2013



Chapter 11: File-System Implementation

11.1 File-System Structure

11.4 Allocation Methods

11.5 Free-Space Management

11.7 Recovery

11.10 Summary





Objectives

- To describe the details of implementing local file systems and directory structures.
- To describe the implementation of remote file systems.
- To discuss block allocation and free-block algorithms and trade-offs.





11.1

File-System Structure

Overview

- File structure:
  - Logical storage unit.
  - Collection of related information.

- **File system** resides on secondary storage (disks)
  - Provided user interface to storage, mapping logical to physical.
  - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily.



File-System Structure

- Disk provides in-place rewrite and random access:
  - I/O transfers performed in **blocks** of **sectors** (usually 512 bytes).
- **File control block** – storage structure consisting of information about a file.
- **Device driver** controls the physical device.

- File system organized into layers (next slide).





File-System Structure

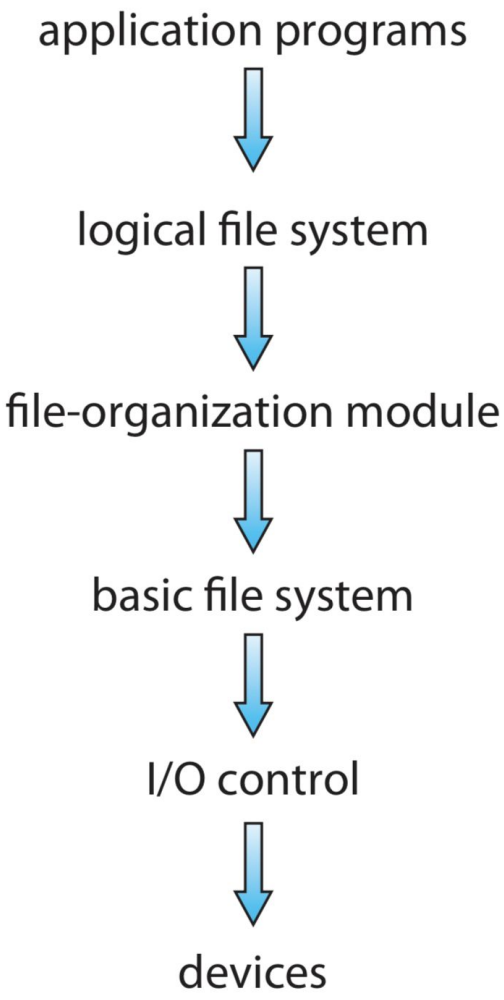


Figure 11.1 Layered file system.

- **Logical file system** manages metadata information includes all the file-system structure, except the actual data.
- **File-organization module** understands files, logical address, and physical blocks.
- **Basic file system** to issue generic commands to the appropriate device driver to read/write physical blocks on the disk.
- **I/O control** consists of device drivers and interrupt handlers to transfer information between main memory and the disk system.



File-System Structure



- **File-organization module** understands files, logical address, and physical blocks.
  - Translates logical block # to physical block #
  - Manages **disk allocation** and **free space**.

Figure 11.1 Layered file system.



Allocation Methods

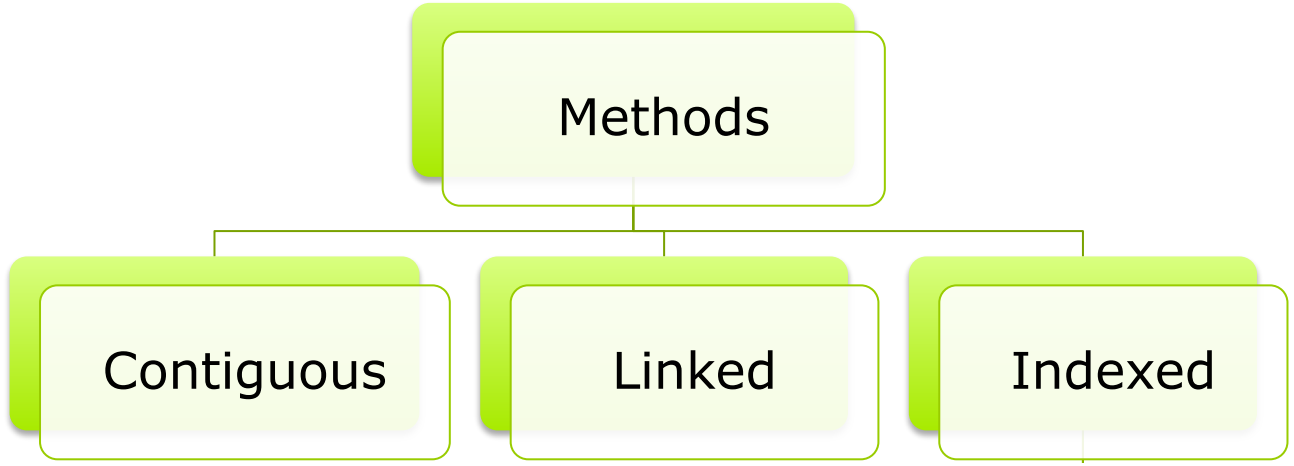
Overview

- A disk is a direct-access device and thus gives us flexibility in the implementation of files.
- The main issue is :
  - ✔ how to allocate space to these files so that disk space is utilized effectively, and
  - ✔ files can be accessed quickly.
- An **allocation method** refers to how disk blocks are allocated for files.



Allocation Methods

- Three major methods of allocating disk space are in wide:



Extent-based system

Index Block Structure



Allocation Methods

(a) Contiguous allocation

- Each file occupies a set of contiguous blocks.
- Best performance in most cases.
- Simple – only starting location (block #) and length (number of blocks) are required.
- Problems include:
  - Finding space for file,
  - Difficult file extension,
  - Knowing file size (max size),
  - External fragmentation,
  - Need for compaction.

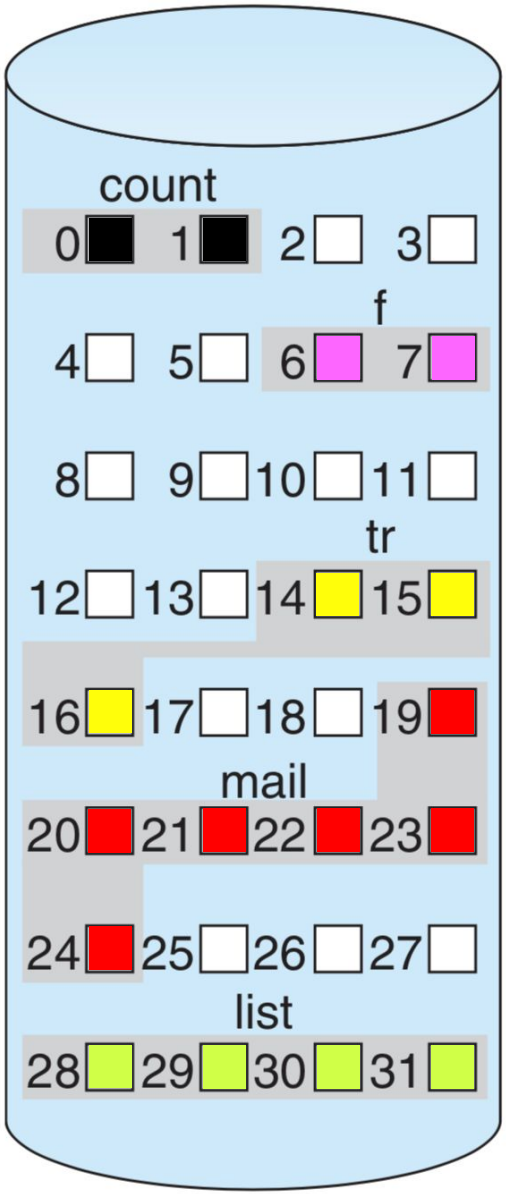


Allocation Methods

Example 1 :

(Page: 506)

Mapping from logical to physical.



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Figure 11.5 Contiguous allocation of disk space.



Allocation Methods

Example 2 : Contiguous file storage for File *A* and *B*.

Free Space	File A Record 1	File A Record 2	File A Record 3	File A Record 4	File A Record 5	File B Record 1	File B Record 2	File B Record 3	File B Record 4	Free Space	File C Record 1
Slot: 0	1	2	3	4	5	6	7	8	9	10	11

File *A*:

- This file cannot be expanded for new record.
- **Solution**: rewrite into a larger storage area.

File *B*:

- This file can be expanded by only one record that replacing the free space at slot 10.



Allocation Methods

Extent-based systems

- A file will grow slowly over a long period (months or years) must be allocated enough space for its final size, even though much of the spaces will be unused for a long time
  - internal fragmentation.

- **Solution**: Modified Contiguous-Allocation scheme.
  - ✔ A contiguous block of space is allocated initially.
  - ✔ If not enough, another contiguous block, known as extent, is added.
  - ✔ A file may consists of one or more extent blocks.



Allocation Methods

(b) Linked allocation

- Each file is a linked list of blocks.
- No external fragmentation.
- Each block contains pointer to next block.
- Free space management system called when new block needed.

Allocation Methods

Example 3 :

(Page: 508)

- A file of five blocks start at blocks 9 and continue at block 16, then block 1, then block 10, and finally block 25.
- Each block contains a pointer to the next block.

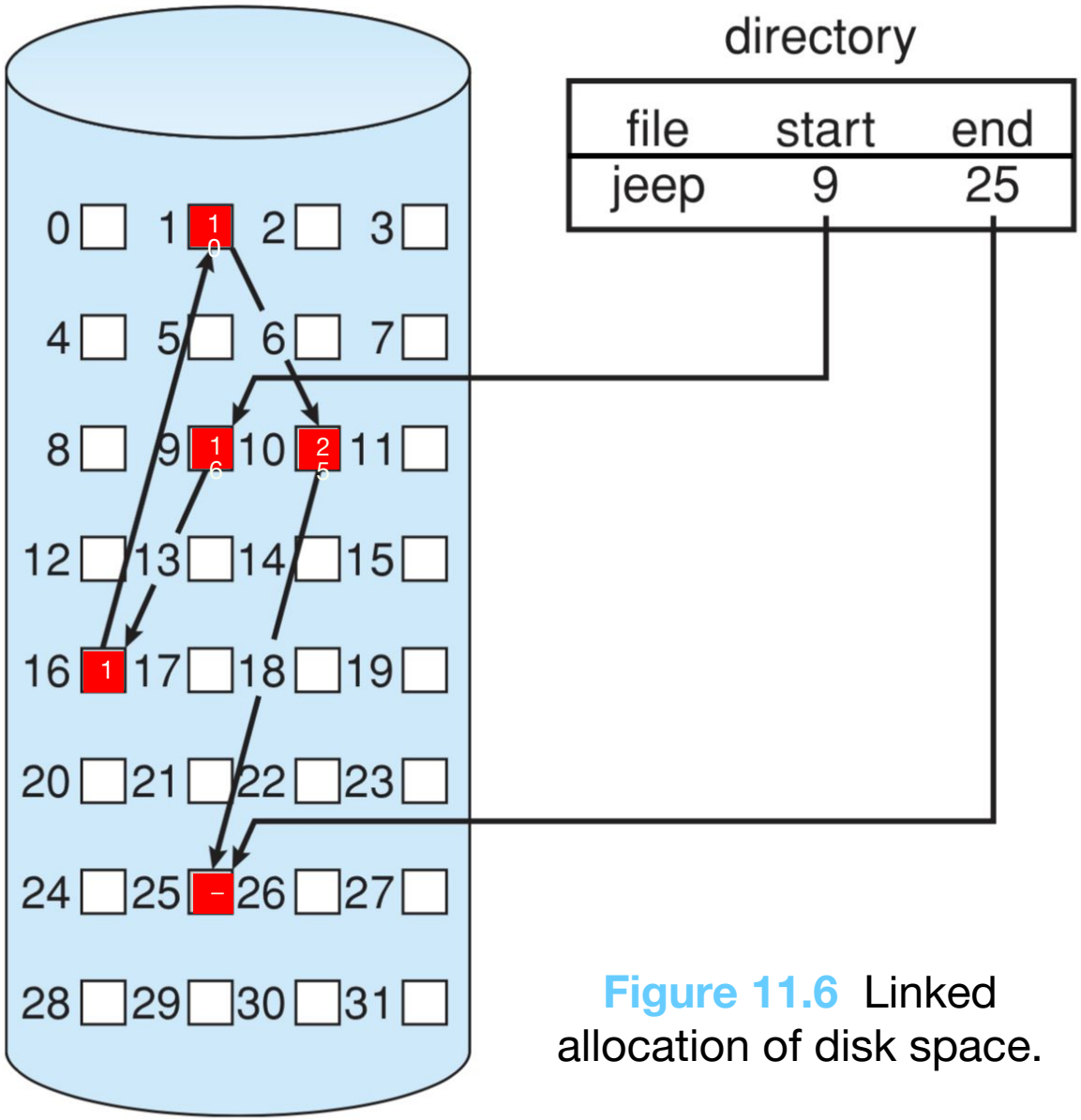
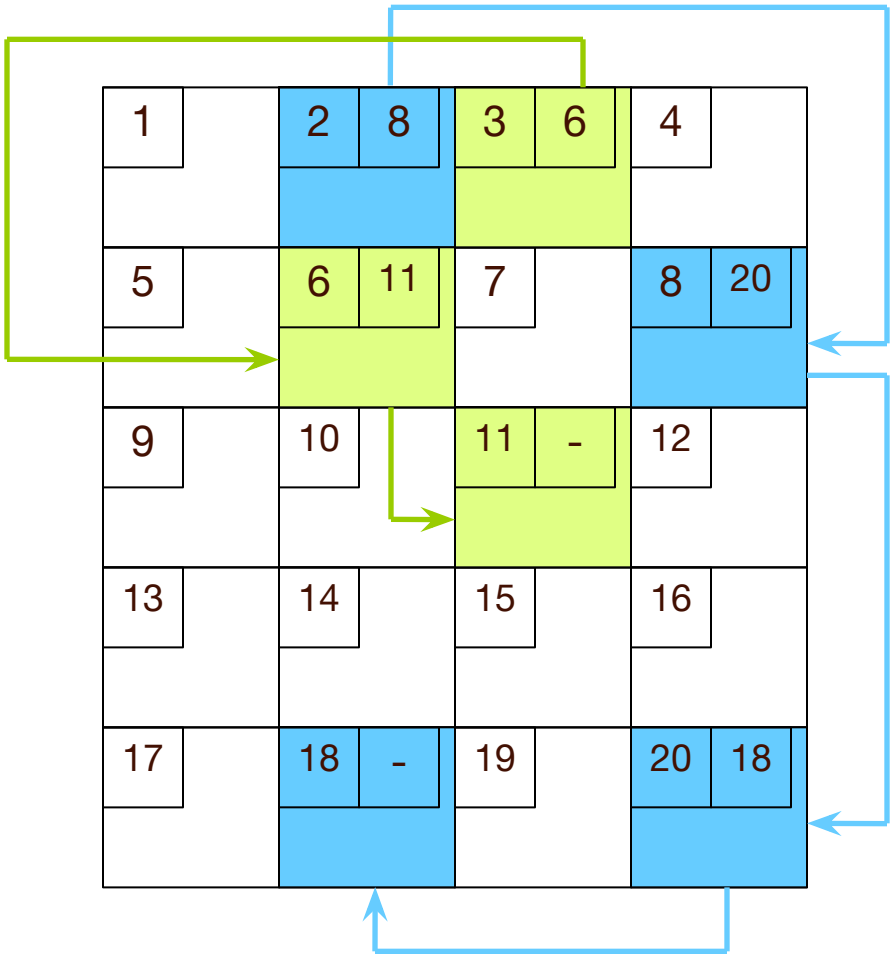


Figure 11.6 Linked allocation of disk space.

Allocation Methods

Example 4 :



- A linked allocation with File A starts at block 2 and continues at block 8, 20, and 18.
- Each block includes its address and a pointer to the next block of the file as well as the data itself.

Directory Entry

File No.	Start	End	No. Extents
A	2	18	3
B	3	11	2



Allocation Methods

- Example 3 and 4 are a sequential-access files.
- To find the i-th block of a file, it starts at the beginning of that file and follow the pointers until the block.

Disadvantages:

- Locating a block can take many I/Os and disk seeks.
- Reliability can be a problem (what if one of the pointers get corrupted?)

Solution: File-Allocation Table (FAT)

- Beginning of volume has table, indexed by block number.
- Much like a linked list, but faster on disk and cacheable.
- New block allocation simple.

Allocation Methods

Example 5 :

(Page: 510)

- FAT structure for a file '*test*' consisting of disk blocks 217, 618, and 339.

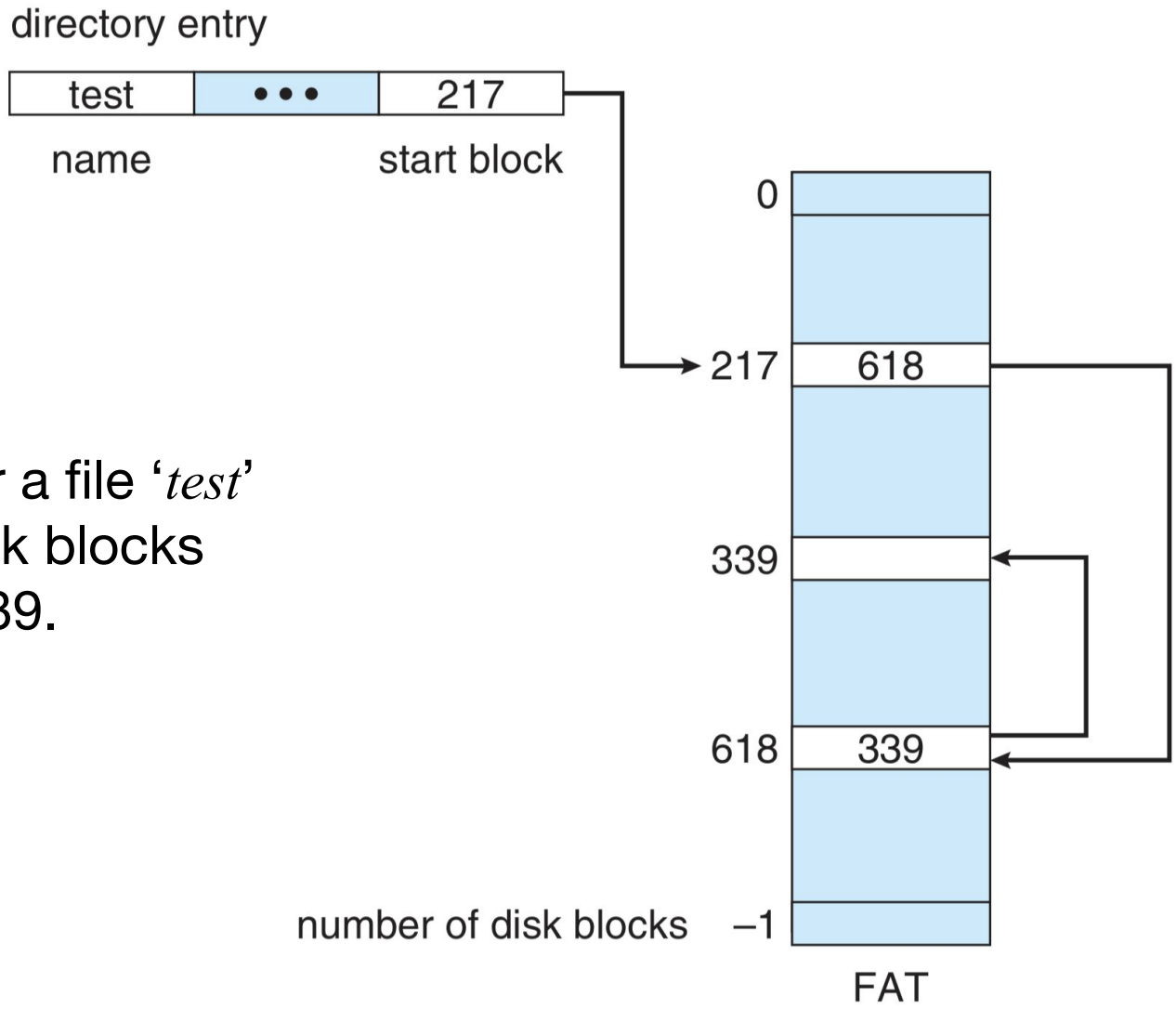


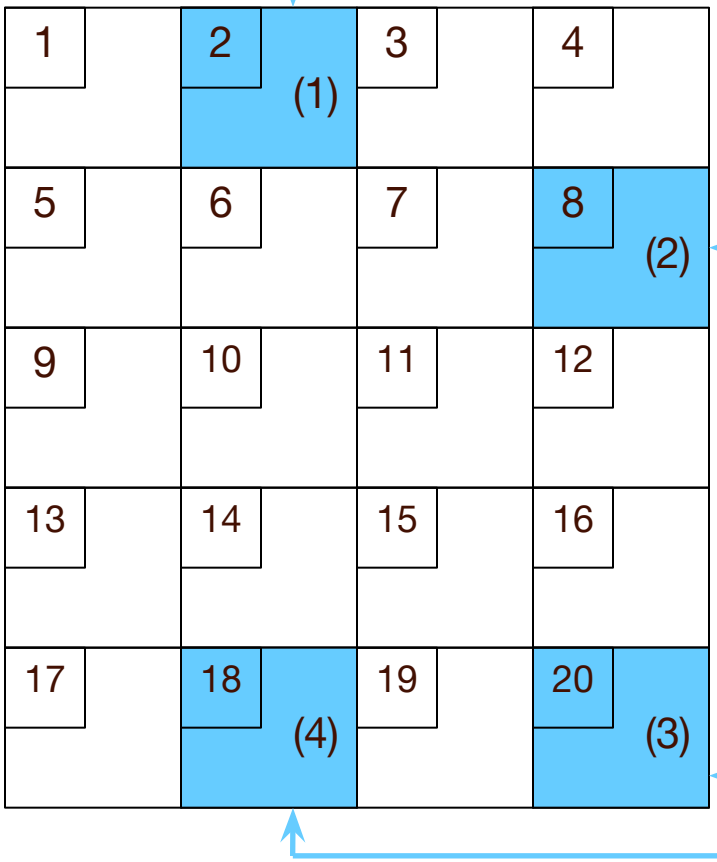
Figure 11.7 File-Allocation Table (FAT).



Allocation Methods

Example 6a :

MS-DOS which uses linked allocation - **FAT**.



Directory Entry

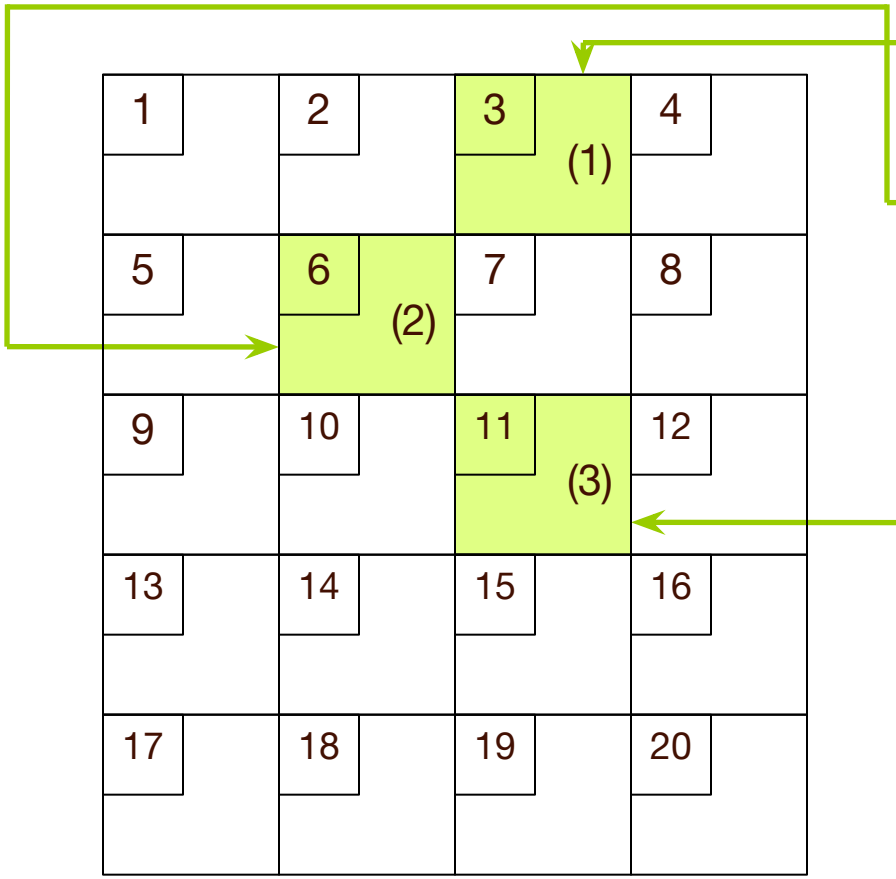
File	Address	Size (KB)	Next
	1	512	
File A (1)	2	512	8
File B (1)	3	512	6
	. . .	. . .	
File B (2)	6	512	11
	7	512	
File A (2)	8	512	20
	. . .	. . .	
File B (3)	11	512	-
	. . .	. . .	
File A (4)	18	512	-
	19	512	
File A (3)	20	512	18



Allocation Methods

Example 6b :

MS-DOS which uses linked allocation - **FAT**.



Directory Entry

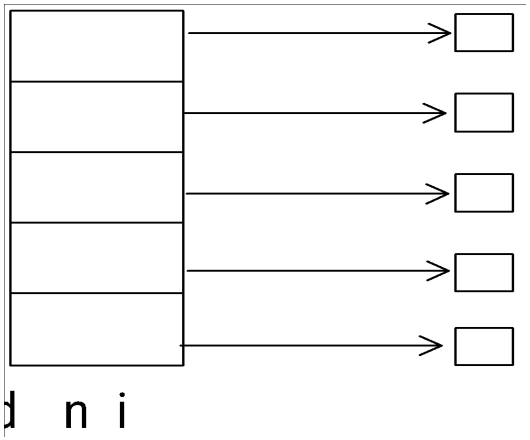
File	Address	Size (KB)	Next
	1	512	
File A (1)	2	512	8
File B (1)	3	512	6
	. . .	. . .	
File B (2)	6	512	11
	7	512	
File A (2)	8	512	20
	. . .	. . .	
File B (3)	11	512	-
	. . .	. . .	
File A (4)	18	512	-
	19	512	
File A (3)	20	512	18



Allocation Methods

(c) Indexed allocation

- Each file has its own **index block**(s) of pointers to its data blocks.
- Logical view:



- Need index table.
- Random access.
- Dynamic access without external fragmentation, but have overhead of **index block**.

Allocation Methods

Example 7 :

(Page: 510)

- The directory contains the address of the index block.
- To find and read the ith block, the pointer in the ith index-block entry is used.

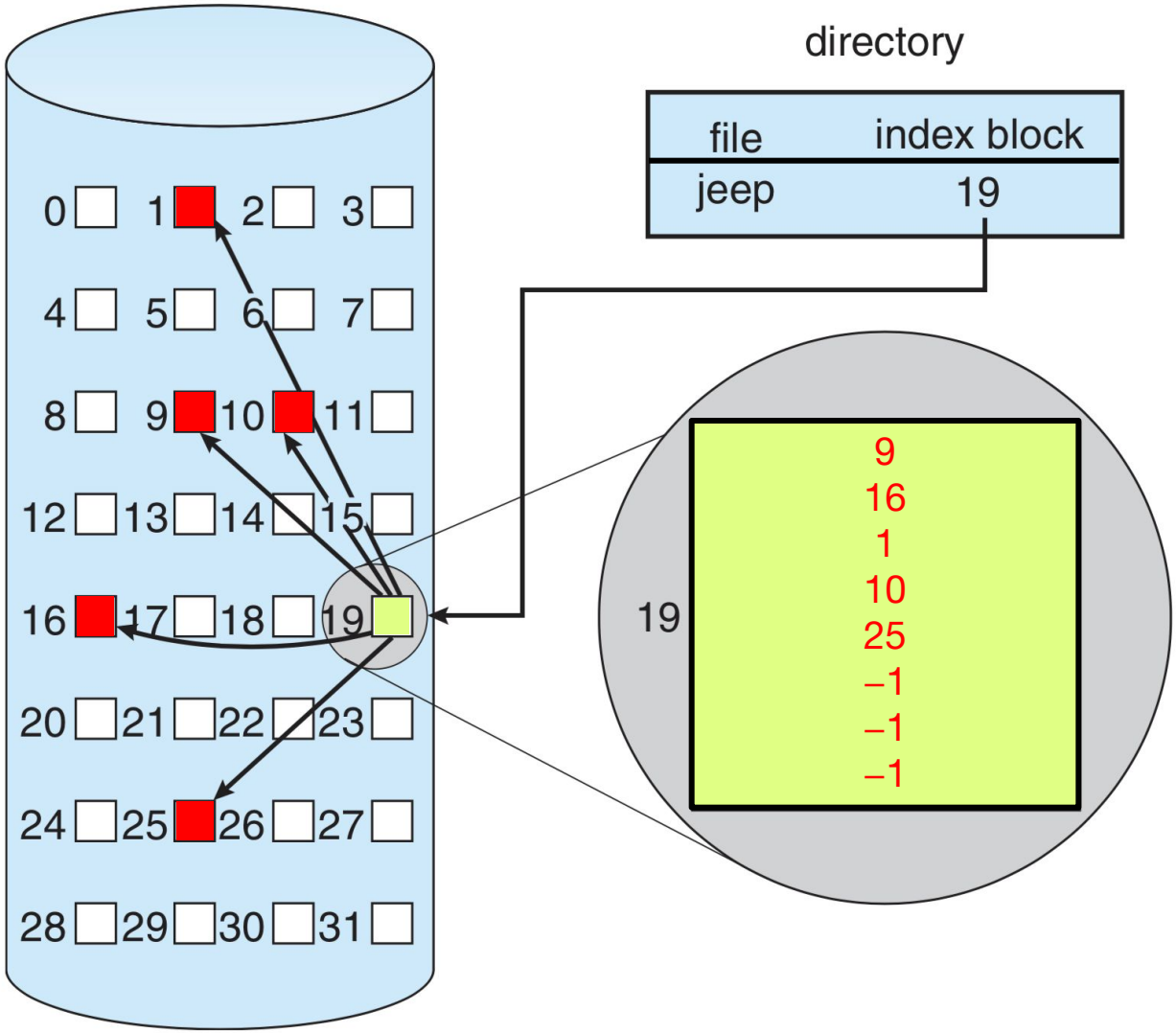
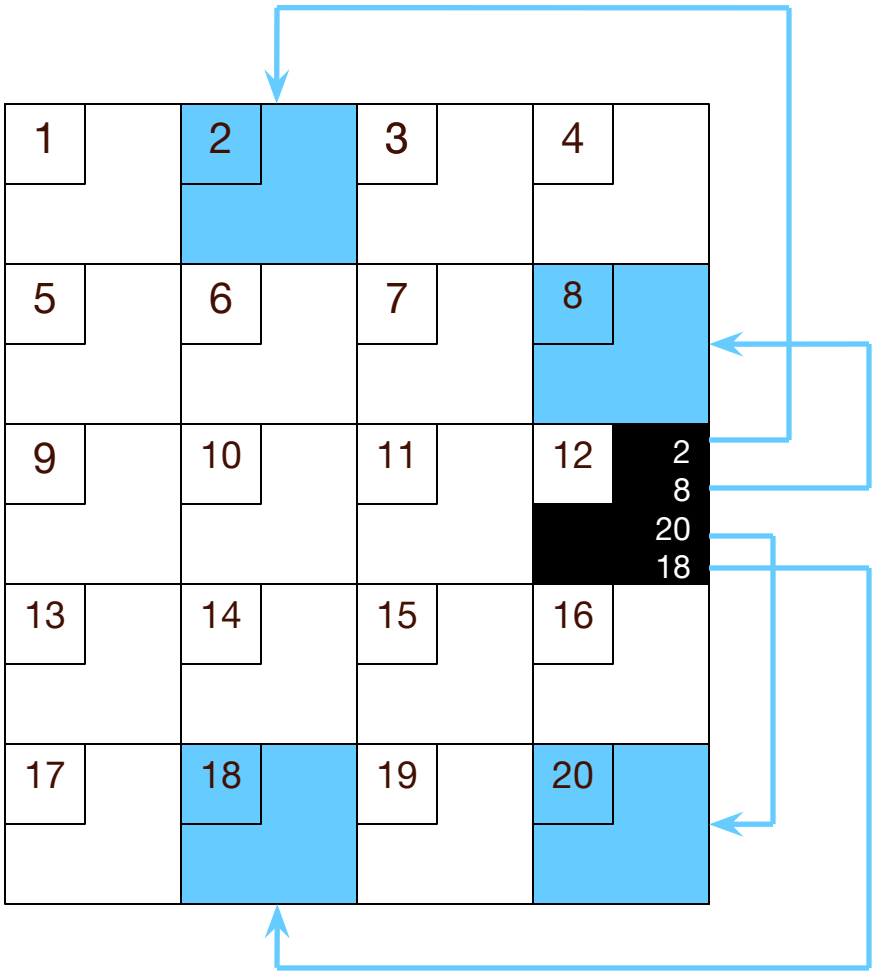


Figure 11.8 Example of Indexed Allocation. Assume each block can only stores 8 block addresses.



Allocation Methods

Example 8a : Indexed allocation for 4 bytes each block.

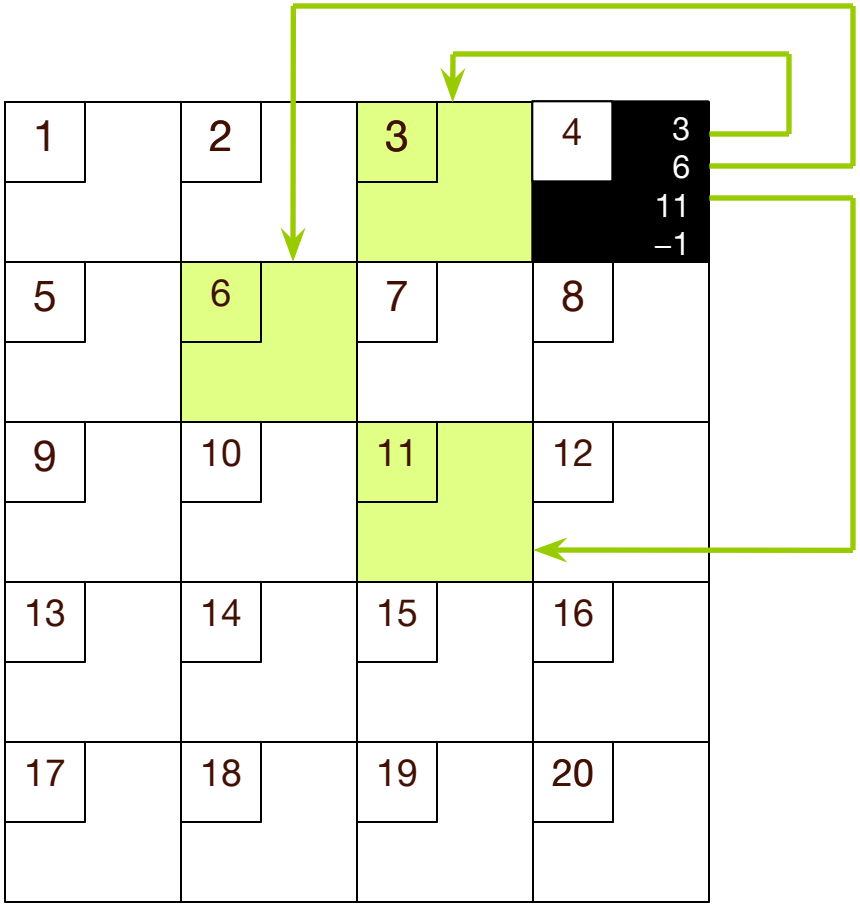


File No.	Index Block
1	12
2	4



Allocation Methods

Example 8b : Indexed allocation for 4 bytes each block.



File No.	Index Block
1	12
2	4

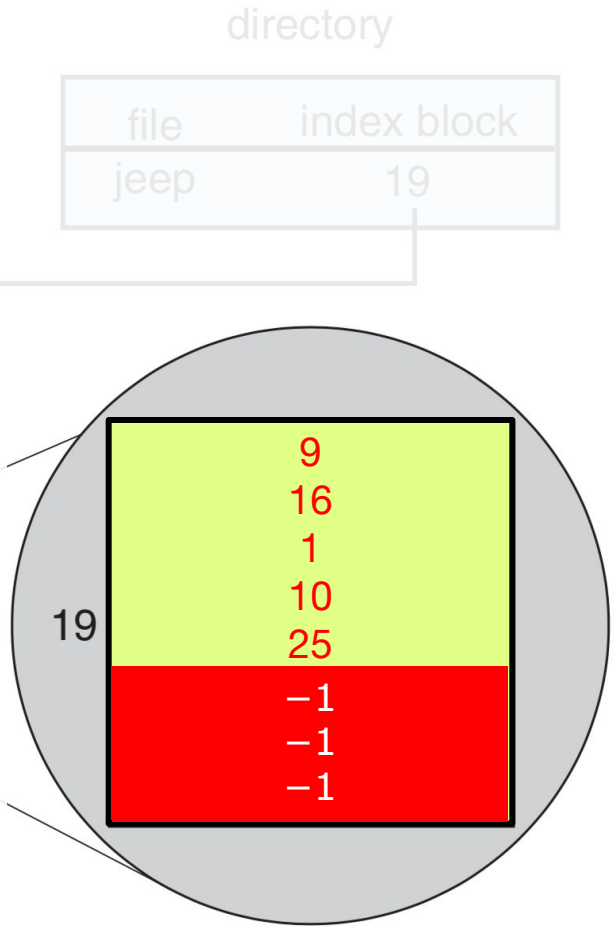
Allocation Methods

Index Block Size Dilemma

- If the index block is:
  - **small** ⇒ it will not be able to hold enough pointers for large files.
  - **large** ⇒ there may be lots of unused space if the file size is small.

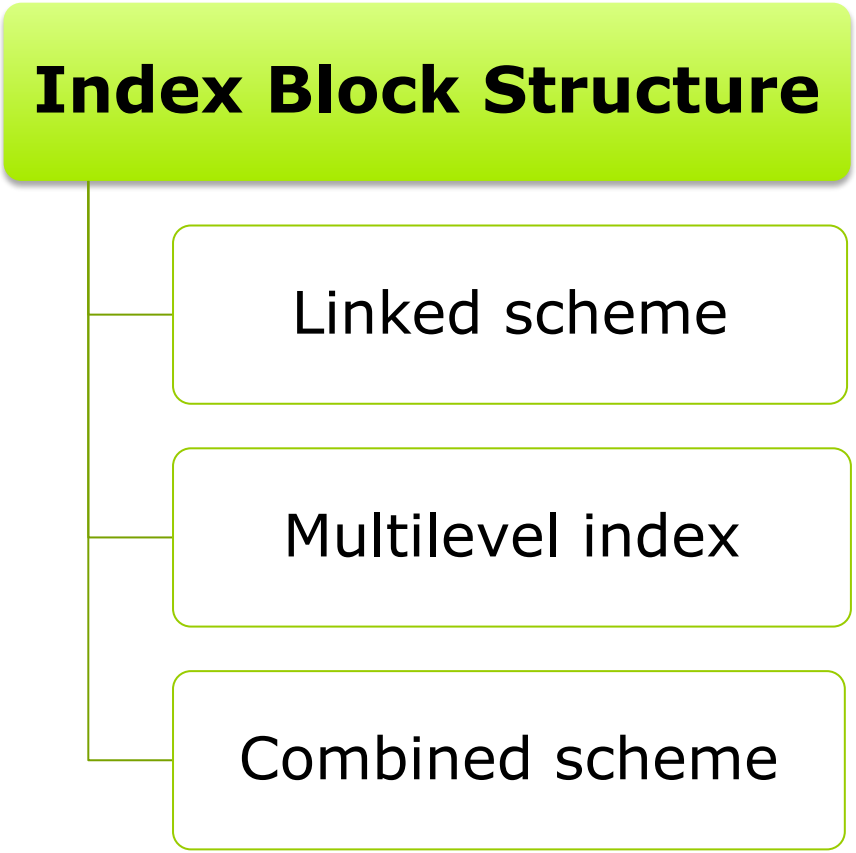
- **Solution**:

  Use **index block structure**





Allocation Methods





Allocation Methods

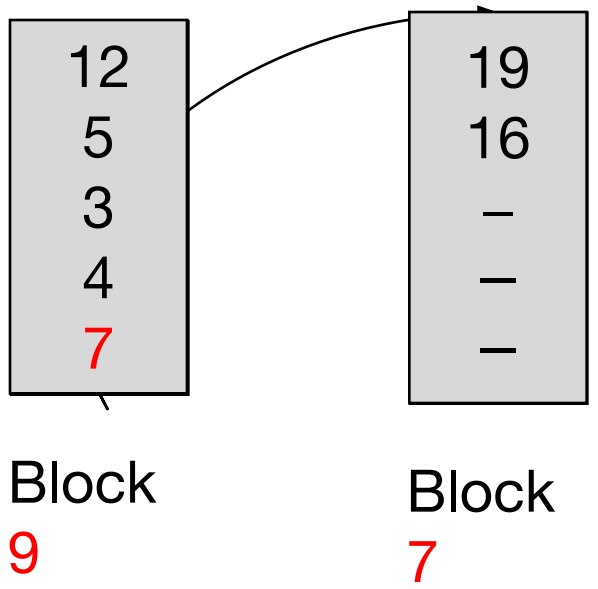
Indexed Allocation: (1) Linked scheme

Example 9 : File blocks □ **9**, 12, 5, 3, 4, **7**, 19, 16.

Assume each block can only stores 5 block addresses.

Directory:

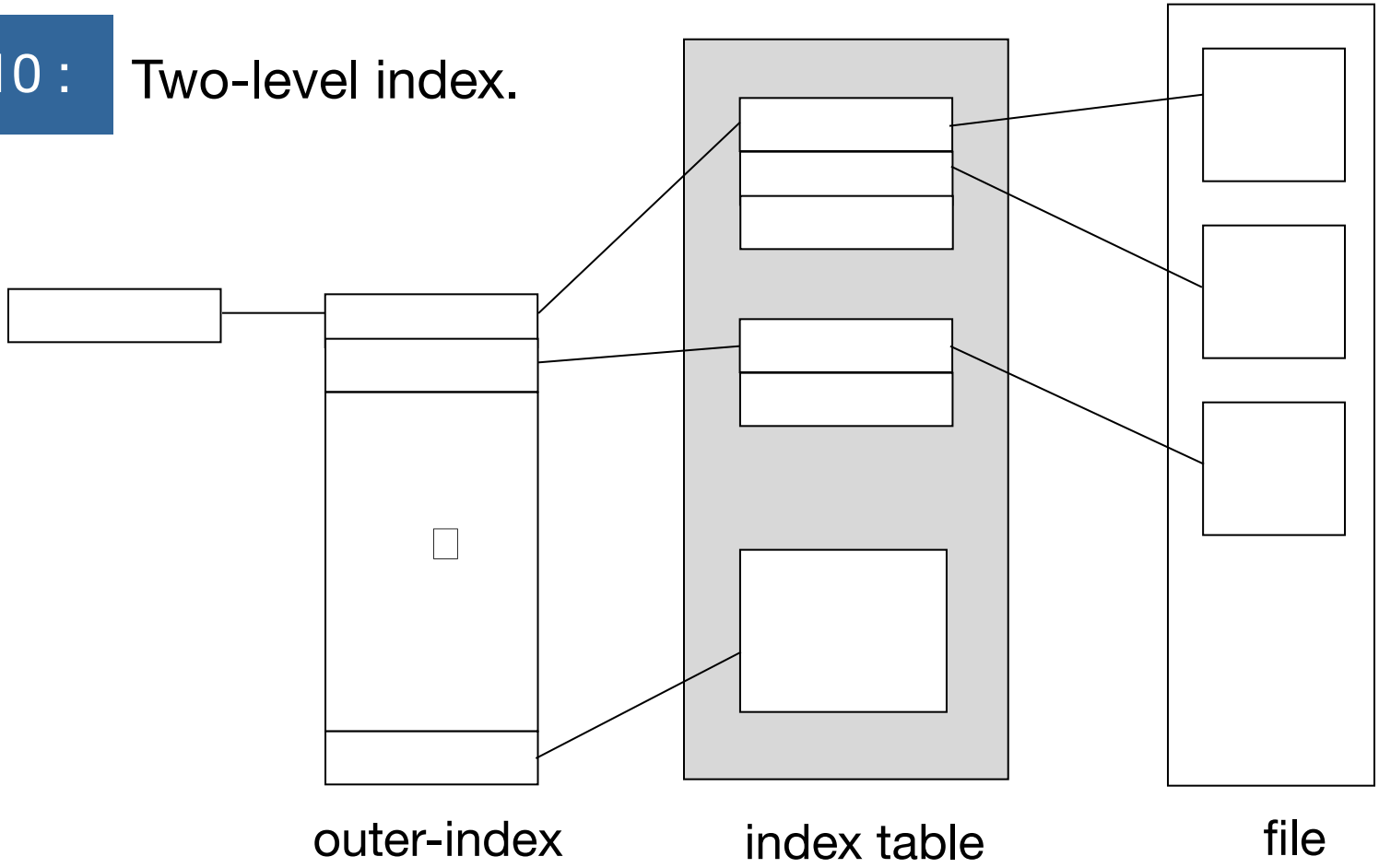
File	Index Block
DATA	9



Allocation Methods

Indexed Allocation: (2) Multilevel index

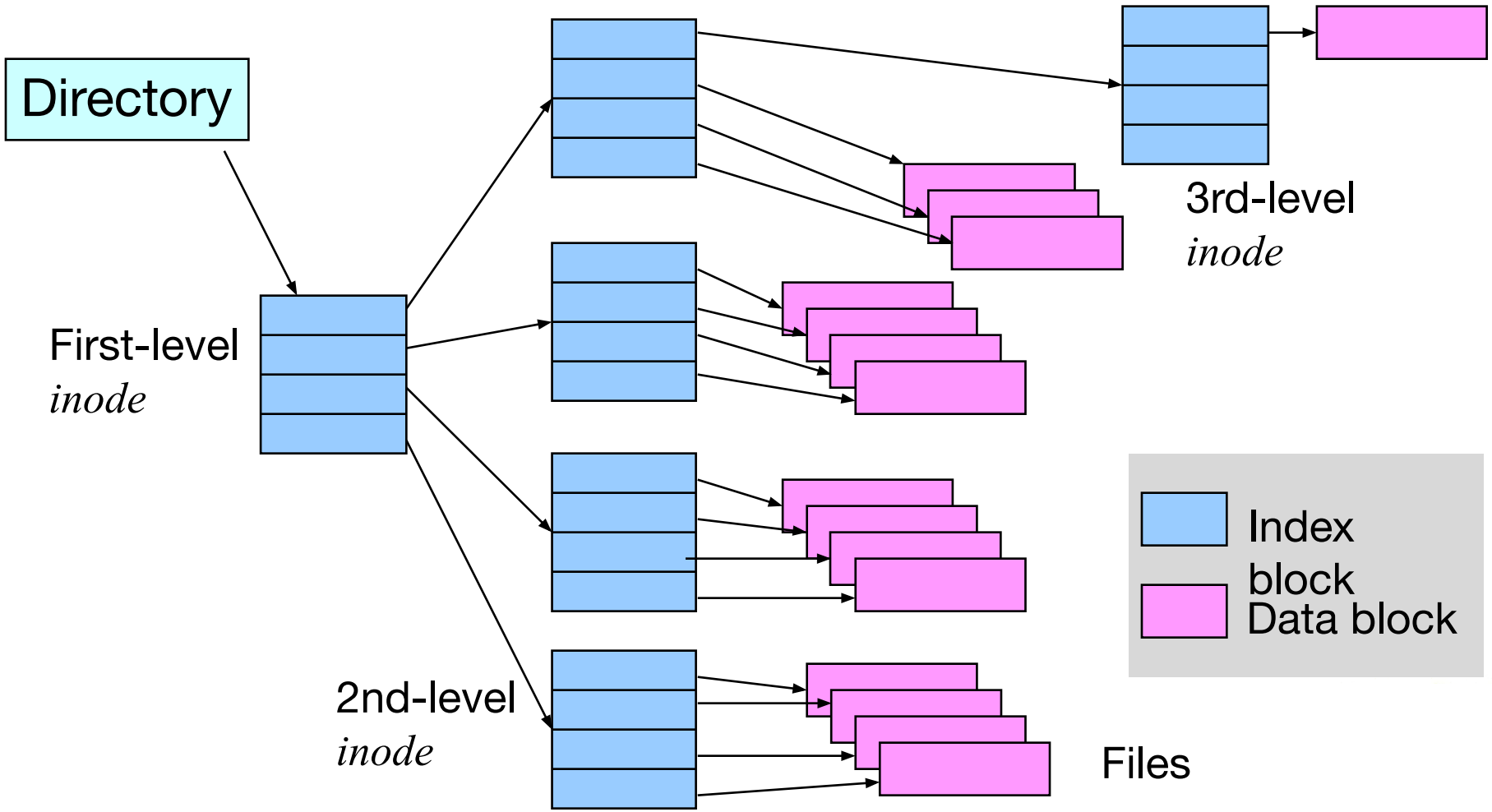
Example 10 : Two-level index.





Allocation Methods

Example 11 : Multilevel index.



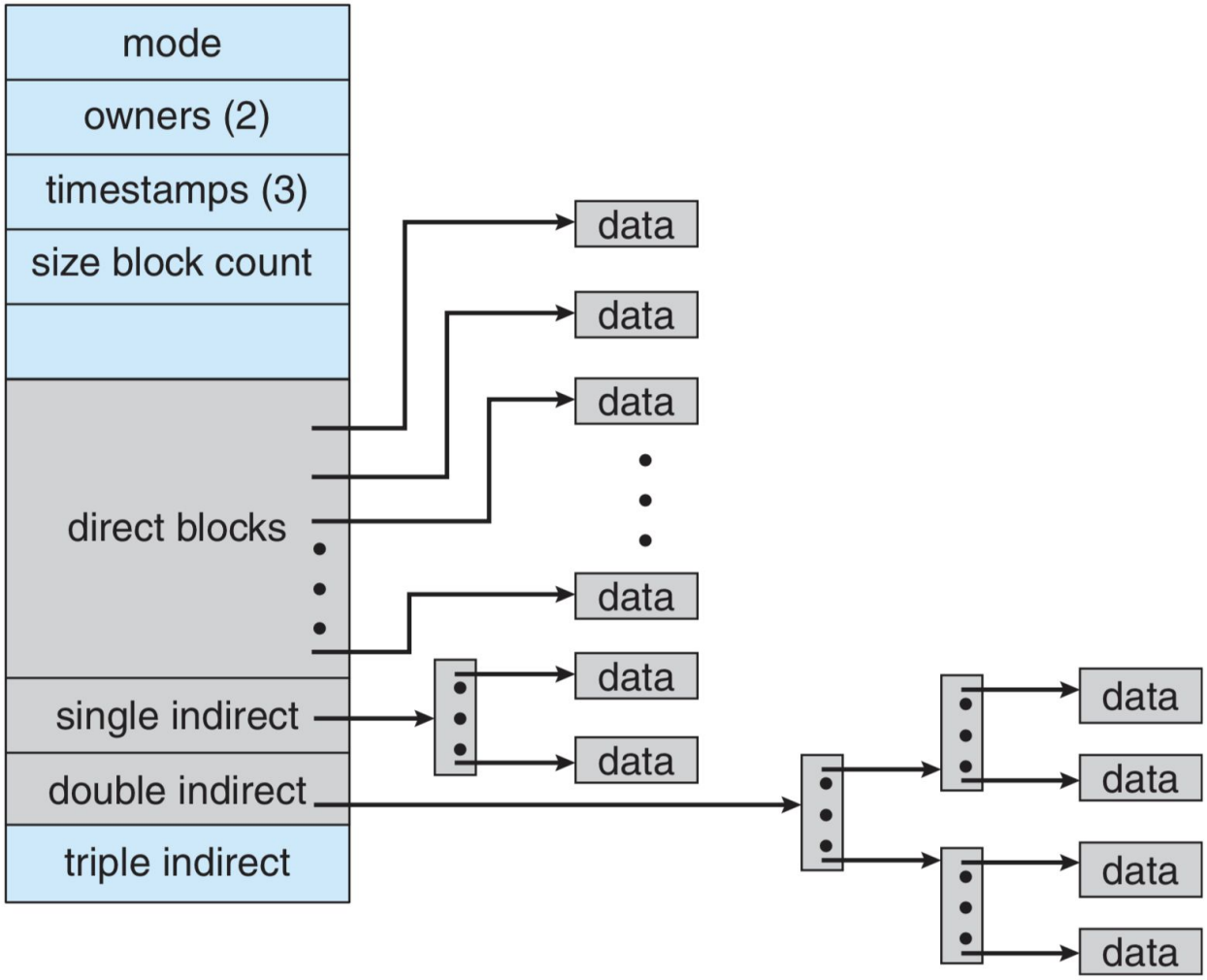
Allocation Methods

Indexed Allocation: (3) Combined scheme

Example 12 :

The UNIX File System (UFS).

- 4K bytes per block, 32-bit addresses.





Allocation Methods

Performance

- Best method depends on file access type.
  - Contiguous great for sequential and random.
- Linked good for sequential, not for random.
- Declare access type at creation select either contiguous or linked.
- Indexed more complex.
  - Single block access could require 2 index block reads then data block read.





11.5

Free-Space Management

Overview

- Since disk space is limited, we need to reuse the space from deleted files for new files, if possible.
- File system maintains **free-space list** to track available **blocks** / clusters.
  - (Using term “block” for simplicity)



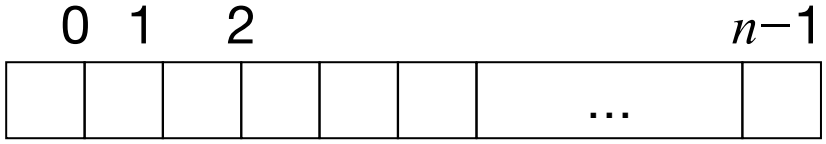


11.5

Free-Space Management

(a) Bit map

- **Bit map** or **bit vector** (n blocks)



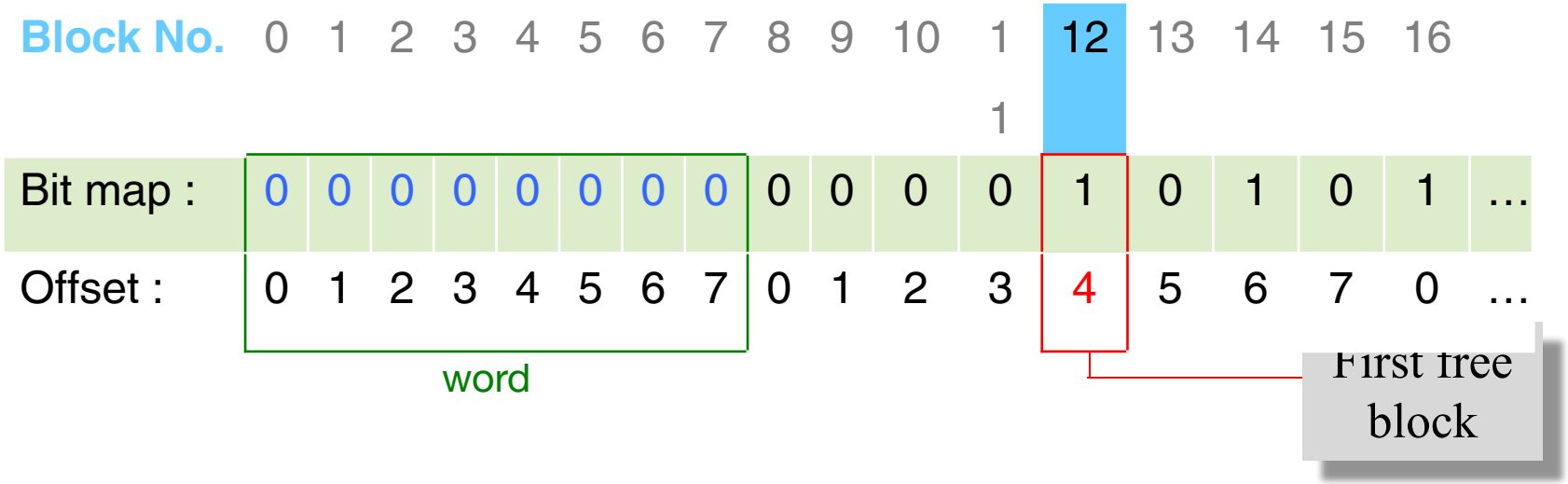
$$\text{bit}[i] = \begin{cases} 1 & \Rightarrow \text{block } [i] \text{ is free} \\ 0 & \Rightarrow \text{block } [i] \text{ is used} \end{cases}$$

- The calculation of the first free block number.
  - The first non-0 word is scanned for the first 1 bit, which is the location of the first free block.



Free-Space Management

Example 13 : Assume there are 8 bits per word and the bit map is as follows:



To find the address of the first free block

= (number of bits per word) * (number of 0-value words) + offset of first 1 bit

= (8 * 1) + 4 = 12



Allocation Methods

Exercise 11.1 : Given a bit map value (0 □ used; 1 □ unused/free).

Block No.:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	…
			1 KB	1 KB					1 KB		1 KB		1 KB				
Bit Map:	0	0	1	1	0	0	0	0	1	0	1	0	1	0	1	0	…

- List the block addresses needed by a file with size 5KB for the different block size of file system.

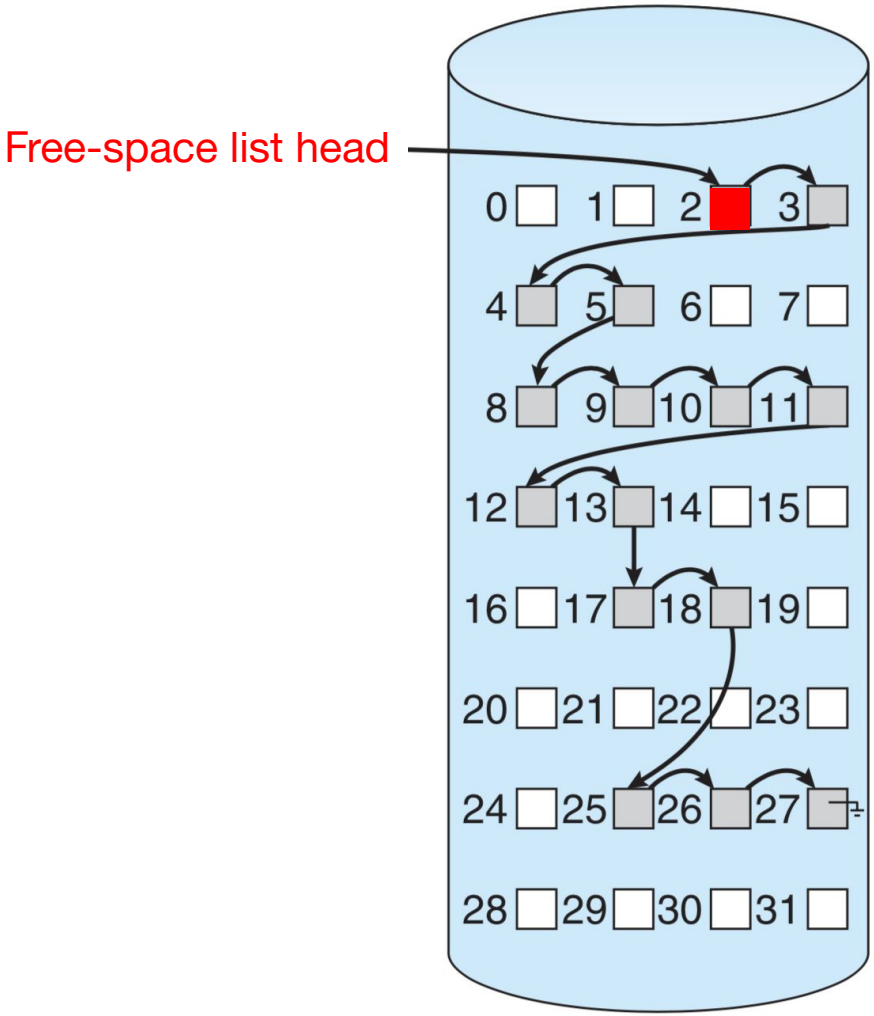
Case(s)	Block size	Address(es)
1	1 KB	2, 3, 8, 10, 12
2	2 KB	2, 3, 8
3	3 KB	2, 3
4	5 KB	2
5	0.5 KB	2, 3, 8, 10, 12, 14, Another 4 addresses



11.5

Free-Space Management

(b) Free list



- Linked list (free list):
  - ✔ Cannot get contiguous space easily.
  - ✔ No waste of space.
  - ✔ No need to traverse the entire list (if # free blocks recorded).

Figure 11.10 Linked free-space list on disk.



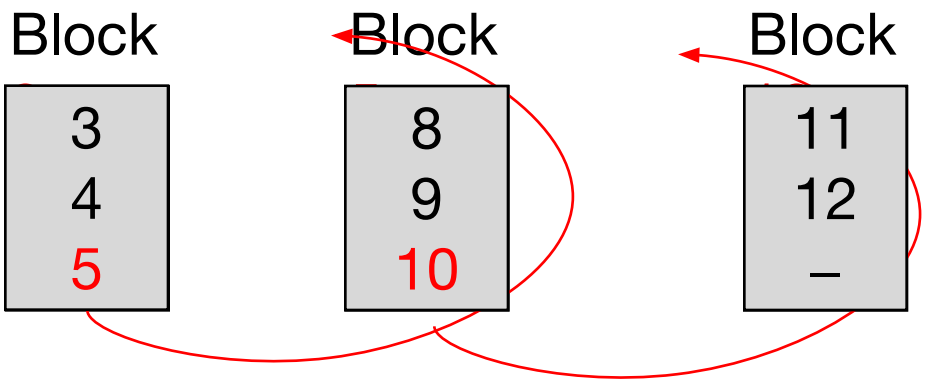
Free-Space Management

(c) Grouping

- Modify **linked list** to store address of next n-1 free blocks in first free block, plus a **pointer** to next block that contains free-block-pointers (like this one)

Example 14 : Free-blocks list □ **2**, 3, 4, **5**, 8, 9, **10**, 11, 12.

Assume each block can only stores 3 block addresses.



Note: block 2, 5 and 10 have become the index block; cannot be used for Data block



Free-Space Management

(d) Counting

- Because space is frequently contiguously used and freed, with contiguous-allocation algorithms, extents, or clustering:
  - Keep address of first free block and count of following free blocks.
  - Free space list then has entries containing addresses and counts.



Recovery

- **Consistency checking** – compares data in directory structure with data blocks on disk, and tries to fix inconsistencies.
  - ✔ Can be slow and sometimes fails.
- Use system programs to **backup** data from disk to another storage device (magnetic tape, other magnetic disk, optical).
- Recover lost file or disk by **restoring** data from backup.



Summary

- The **file system** resides permanently on secondary storage disk.
- Physical disks may be segmented into partitions to control media use and to allow multiple file systems.
- Any file-system type can have different structures and algorithms.
- The various files can be allocated space on the disk in three ways: **contiguous**, **linked**, or **indexed** allocation.





Summary

- Free-space allocation methods also influence the efficiency of disk-space use, the performance of the file system, and the reliability of secondary storage. **bit map**, **linked lists**, **grouping**, and **counting**.





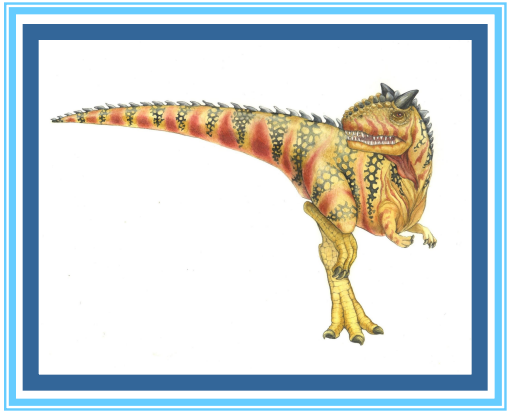
Exercises

From Silberchatz, Operating System Concepts
Chapter 11 Exercises (page: 533 - 534)

- 11.2
- 11.7
- 11.8
- 11.9
- 11.10



End of Chapter 11



Silberschatz, Galvin and Gagne ©2013